Plataforma de gestión de torneos de fútbol formativo: propuesta de proyecto final

Nicolás Molano Barón · Luis Gabriel Peraza Muñoz · Rafael Otero Erazo · Sebastián Clavijo Tocasuchyl · Nicolás Gómez Lozano
Ingeniería de Software II — Departamento de Ingeniería de Sistemas e Industrial
Universidad Nacional de Colombia, Bogotá, Colombia

***Resumen*—Los torneos de fútbol formativo se administran, en la mayoría de los casos, con hojas de cálculo, grupos de mensajería y papel, lo que produce errores de digitación, jugadores sin habilitar, dobles reservas de cancha y tablas desactualizadas. Este documento propone una aplicación web que centraliza el ciclo completo del torneo, desde la inscripción y la carnetización hasta la publicación de resultados, y reparte el trabajo entre organizador, delegados de equipo y árbitros. Se describen el problema, la solución, los actores, las funcionalidades principales, los roles del grupo y las tecnologías previstas.**

***Palabras clave*—gestión de torneos, aplicación web, PostgreSQL, seguridad a nivel de fila, PWA, React**

I. Introducción: el problema

Los torneos de fútbol formativo (escuelas, ligas inter-clubes, categorías infantiles y juveniles) se siguen administrando, en la mayoría de los casos, con una mezcla de hojas de cálculo, grupos de WhatsApp y papel. Un torneo mediano puede tener fácilmente 30 o 40 categorías, más de 250 equipos y unos cuantos miles de jugadores inscritos. Con esos números, el organizador termina haciendo a mano cosas que no deberían hacerse a mano: revisar documentos de cada jugador para expedir su carnet, cuadrar quién pagó y quién debe, armar el fixture en un Excel con macros que solo una persona sabe tocar, negociar por chat la cancha y la hora de cada partido, y pasar los resultados de las planillas físicas a una tabla de posiciones que se publica como imagen en redes.

El resultado es lo esperable: errores de digitación, jugadores que juegan sin estar habilitados, dobles reservas de cancha, tablas desactualizadas y una organización que vive apagando incendios los viernes por la noche.

II. Cómo lo vamos a resolver

Queremos construir una aplicación web que centralice el ciclo completo del torneo, desde la inscripción hasta la publicación de resultados, y que reparta el trabajo entre quienes realmente tienen la información. La idea de fondo es que el organizador deje de ser el cuello de botella: el delegado de cada equipo inscribe a sus jugadores y sube sus documentos, propone la cancha y el horario de su próximo partido y registra el marcador; el rival lo confirma; el árbitro reporta su disponibilidad; y la organización solo revisa, aprueba y resuelve conflictos.

Para que eso funcione sin que se vuelva un caos, las reglas del torneo tienen que vivir en el sistema y no en la memoria de una persona. Por eso la lógica crítica (que no haya dos partidos en la misma cancha y franja, que un resultado aprobado no se pueda alterar sin dejar rastro, que el sorteo de grupos no junte equipos del mismo club si se puede evitar) se implementa como restricciones e integridad en la base de datos, y no solo como validaciones en la interfaz.

El sistema se diseña desde el inicio como multi-torneo y multi-organización, de modo que la misma plataforma pueda ser usada por distintas ligas, cada una con sus categorías, su reglamento y su marca, sin que los datos de una se mezclen con los de otra.

A. Base preexistente y alcance del semestre

Conviene decirlo desde el principio: el proyecto parte de una base construida por uno de los integrantes, Nicolás Molano Barón, por fuera de la universidad y antes de este curso, que ya cubre la inscripción, la carnetización y la tesorería y que está en uso en un torneo real. Esa base es la propiedad intelectual de su autor y se usa en el curso con su autorización. El trabajo del grupo durante el semestre consiste en extender esa base con un módulo nuevo, que se acordará con el docente en las primeras semanas, aplicando en él el proceso completo de ingeniería de software que pide la asignatura: requisitos, diseño, implementación, pruebas y entrega.

III. Actores

- *Administrador de la plataforma.* Crea organizaciones y torneos, define la configuración global y tiene visibilidad sobre todo.
- *Organizador del torneo.* Es el usuario principal. Configura categorías, canchas y franjas horarias, valida documentos y expide carnets, lleva la tesorería, aprueba la programación de cada jornada y los resultados, y genera las piezas de comunicación.
- *Delegado o gerente deportivo de equipo.* Inscribe jugadores y cuerpo técnico, sube documentos, consulta el estado de cuenta de su equipo, programa sus partidos en acuerdo con el rival y reporta resultados. Un gerente puede manejar varios equipos del mismo club.
- *Árbitro.* Registra su disponibilidad por jornada y consulta los partidos que le fueron asignados.
- *Público general.* Consulta programación, tablas de posiciones y goleadores sin iniciar sesión, a través de una vista pública restringida a datos no sensibles.

IV. Funcionalidades principales

1) *Inscripción y carnetización.* Registro de jugadores por equipo y categoría con carga de documentos, flujo de validación (pendiente, aprobado, rechazado con motivo), control de duplicados y generación del lote listo para impresión de carnets. Toda validación queda auditada: quién la hizo, cuándo y qué cambió.
2) *Tesorería.* Valor de inscripción por categoría con excepciones por equipo, registro de abonos, estados de cuenta por club (un club puede agrupar varios equipos en una sola cuenta) y reportes descargables.
3) *Configuración de la competición.* Copas, fases, grupos y reglas de clasificación por categoría. Sorteo de grupos aleatorio con restricción flexible de club y ajuste manual arrastrando equipos.
4) *Fixture.* Generación automática del calendario por el método del círculo (round-robin), con balance de local y visitante, ida y vuelta cuando aplique y manejo de grupos impares.
5) *Programación colaborativa de partidos.* La organización abre una jornada con una ventana de autogestión; el equipo local propone cancha y franja, la propuesta bloquea el cupo por tiempo limitado, el visitante acepta y queda reservado. Al cerrar la ventana, lo que quedó sin programar se asigna automáticamente y la organización aprueba. Exclusión estricta de dobles reservas.
6) *Árbitros.* Disponibilidad por jornada y asignación asistida: al asignar un juez a un partido solo aparecen los que están libres en esa franja.
7) *Resultados y tablas.* El delegado carga marcador y goleadores, el rival confirma, el resultado entra en revisión y cuenta provisionalmente en la tabla; la organización lo verifica contra la planilla física y lo aprueba. Posiciones, goleadores y valla menos vencida se calculan solos, incluyendo retiros, W.O. y cambios de fase.
8) *Comunicación.* Generación de imágenes de programación y resultados con la marca de la organización, listas para redes sociales, y notificaciones push a delegados desde la aplicación instalada en el celular.

V. Roles de los integrantes

Vamos a trabajar con Scrum, en sprints de dos semanas, con una revisión al cierre de cada uno con el docente como parte interesada. Los roles quedan así:

- *Product Owner: Nicolás Molano Barón.* Es el autor de la base preexistente y quien tiene el contacto con el organizador real que sirve de piloto, así que es quien mejor conoce el problema. Mantiene y prioriza el product backlog, escribe las historias de usuario con sus criterios de aceptación, resuelve dudas del equipo sobre el negocio y acepta o rechaza lo entregado en cada sprint.
- *Scrum Master: Luis Gabriel Peraza Muñoz.* Facilita las reuniones (planificación, daily, revisión y retrospectiva), cuida que el equipo respete el proceso, quita impedimentos y lleva el tablero y las métricas del sprint. También desarrolla, pero su responsabilidad principal es que el equipo funcione.
- *Developers: Rafael Otero Erazo, Sebastián Clavijo Tocasuchyl y Nicolás Gómez Lozano, junto con Luis Gabriel Peraza Muñoz.* Equipo autoorganizado y multifuncional: entre todos cubren backend y base de datos, frontend, lógica de competición y pruebas. En cada planificación se reparten las historias según carga y afinidad, no por un cargo fijo, y cada historia se da por terminada solo cuando pasa revisión de código, pruebas automatizadas y la aceptación del Product Owner.

El Product Owner también revisa e integra técnicamente los cambios al repositorio, porque la base es suya y está en uso real; esa doble función se declara desde ya para que no se confunda con el rol de desarrollador.

VI. Tecnologías y frameworks

- *Frontend: React con un framework de renderizado en servidor (App Router, componentes de servidor y acciones de servidor), TypeScript y Tailwind CSS para los estilos.* La aplicación se empaqueta como PWA (manifiesto, service worker y Web Push con VAPID).
- *Backend y datos: PostgreSQL como única fuente de verdad, con seguridad a nivel de fila (RLS) para aislar organizaciones y roles, triggers y restricciones para las reglas críticas, y migraciones SQL versionadas en el repositorio.* Autenticación basada en tokens JWT y almacenamiento de objetos para documentos e imágenes.
- *Validación y tipos: esquemas con Zod en los límites del sistema y tipos de base de datos generados automáticamente desde el esquema.*
- *Pruebas: Vitest para la lógica pura (fixture, sorteo, clasificación) y Playwright para las pruebas de extremo a extremo de los flujos por rol.*
- *Proceso: Git con ramas por funcionalidad y revisión por solicitud de cambios, integración continua que corre pruebas y verificación de tipos en cada cambio, y despliegue continuo en una plataforma serverless con entornos de previsualización por rama.*

La meta para el cierre del semestre es tener el módulo nuevo funcionando de punta a punta sobre datos de un torneo real, con los cuatro roles operando sobre la misma plataforma.